

Bases de Datos I

Trabajo Práctico Integrador 5: Transacciones y control de concurrencia

Alumno: Gómez Julian Daniel

DNI: 38844753

Link al repositorio con capturas y scripts SQL: [Branch dev/julian - Repositorio GitHub](#)

Descripción:

Diseñar y ejecutar pruebas que muestren el comportamiento de la base ante accesos simultáneos, implementando transacciones y evaluando bloqueos, deadlocks y niveles de aislamiento.

Nota de alcance. Aquí se observa cómo los índices afectan la contención (bloqueos/esperas) en acceso simultáneo; la comparación de tiempos crudos aproximados con/sin índice ya se realizó en Etapa 3.

Objetivos

- Implementar transacciones en operaciones críticas.
- Comparar niveles de aislamiento en MySQL.
- Diagnosticar y manejar bloqueos y deadlocks.
- Medir impacto de índices en entornos concurrentes.

Metodología

1. Simulación con dos sesiones MySQL para generar bloqueos y deadlocks.
2. Comparación de READ COMMITTED y REPEATABLE READ.
3. Uso **opcional** de mysqlslap para carga concurrente.
4. Implementación de retry ante deadlock en Java o SQL (procedimiento almacenado).
5. Elaborar un breve informe (5–10 líneas) con las observaciones sobre concurrencia y transacciones.
6. Interacción con IA: usarla como apoyo para interpretar los resultados de las pruebas de concurrencia o sugerir pasos adicionales, dejando evidencia del intercambio.

Entregables

- Guion de pruebas SQL.
- Resultados de mediciones.
- Código cliente (Java) o procedimiento SQL con transacciones y manejo de errores (incluyendo estrategia de retry si corresponde).
- Evidencia del proceso.

Resolución:

1) Simulación de Deadlock usando dos sesiones

Se crearon dos sesiones distintas con el mismo usuario Host por razones de practicidad en el workbench 8.0 de MySQL:

- Sesión A (con conexión ID= 10)

The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' tree with 'biblioteca' selected. The main editor shows a SQL script for 'SCRIPT A: Simulación deadlock transaccional'. The script includes the following steps:

```
1 -- SCRIPT A: Simulación deadlock transaccional --
2 SELECT CONNECTION_ID();
3
4 SET autocommit = 0;
5 START TRANSACTION;
6
7 -- A1) Bloquea primero una fila de LIBRO (por ejemplo id 1)
8 UPDATE Libro
9 SET titulo = CONCAT(titulo, '*A')
10 WHERE id_Libro = 1;
11
12 ROLLBACK
13
14 -- (No se realiza el COMMIT todavía)
15
16
17 -- A2) Ahora A intenta bloquear la misma fila de FICHA que tiene B
18 UPDATE FichaBibliografica
19 SET estanteria = 'ZA'
20 WHERE id_ficha = 2;
21 -- Esto quedará esperando (B tiene el lock).
22
23
```

The 'Result Grid' at the bottom shows the output of the first query, 'CONNECTION_ID()', which is '10'.

#	Time	Action	Message
1	21:13:21	SELECT CONNECTION_ID() LIMIT 0, 1000	1 row(s) returned

- Sesión B (con conexión ID = 16)

MySQL Workbench

facultad (biblioteca) - Warning.X facultad - Warning - not su... x

File Edit View Query Database Server Tools Scripting Help

Navigator: MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Performance Schema Setup

Query 1: deadlock_sesion_B*

```
1  -- SCRIPT B: Simulación deadlock transaccional --
2  • SELECT CONNECTION_ID();
3
4  • SET autocommit = 0;
5  • START TRANSACTION;
6
7  -- B1) Bloquea primero una fila de FICHA (por ejemplo id_ficha 2)
8  • UPDATE FichasBibliograficas
9    SET estanteria = 'ZB'
10   WHERE id_ficha = 2;
11
12  -----
13
14  -- B2) B intenta bloquear la misma fila de LIBRO que tiene A
15  • UPDATE Libro
16    SET titulo = CONCAT(titulo, ' *B')
17   WHERE id_Libro = 1;
18  -- Acá MySQL detecta el deadlock y CANCELA una de las dos transacciones con error:
19  -- ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
20
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

CONNECTION_ID()
16

Result 1 x

Output

Action Output

#	Time	Action
✓ 1	21:14:31	SELECT CONNECTION_ID() LIMIT 0, 1000

Valores iniciales de la tabla Libro antes de la prueba:

Paso 1: En sesión A, se inicializa una transacción y se desactiva la función autocommit del workbench. Luego se aplica una modificación a Libro con UPDATE (se bloquea el recurso), SIN REALIZAR COMMIT

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following script:

```
1 • USE Biblioteca;
2 • SELECT CONNECTION_ID();
3
4 • SET autocommit = 0;
5 • START TRANSACTION;
6
7 -- Bloqueamos el libro 1 (queda "ocupado" hasta hacer COMMIT/ROLLBACK)
8 • UPDATE Libro
9 SET titulo = CONCAT(titulo, '*LOCK*')
10 WHERE id_Libro = 1;
11
12 -- No se realiza todavía el COMMIT
13 • SELECT CONNECTION_ID() AS connA, 'A: bloqueo activo' AS estado;
14
15 COMMIT
16 -- ROLLBACK
```

The Result Grid shows the output of the script:

connA	estado
10	A: bloqueo activo

The Output window shows the execution log:

#	Time	Action	Message
1	00:53:20	USE Biblioteca	0 row(s) affected
2	00:53:20	SELECT CONNECTION_ID() LIMIT 0, 1000	1 row(s) returned
3	00:53:20	SET autocommit=0	0 row(s) affected
4	00:53:20	START TRANSACTION	0 row(s) affected
5	00:53:20	UPDATE Libro SET titulo = CONCAT(titulo, '*LOCK*') WHERE id_Libro = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
6	00:53:20	SELECT CONNECTION_ID() AS connA, 'A: bloqueo activo' AS estado LIMIT 0, 1000	1 row(s) returned

Paso 2: Se inicia otra transacción en la sesión B y se intenta bloquear la tabla Libro al realizar un UPDATE para modificar una de sus entradas. Como el recurso está siendo bloqueado en la sesión A, se produce el error **deadlock**: una transacción no puede acceder a un mismo recurso que está siendo utilizado (y bloqueado) por otra.

Este mecanismo de bloqueos tiene como propósito preservar la integridad de los datos y evitar inconsistencias en los valores en tiempo de ejecución. El error deadlock sea da justamente por intentar acceder a un recurso bloqueado por otra transacción (que se desbloquean al realizar un COMMIT o ROLLBACK)

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following script:

```
6 • USE Biblioteca;
7
8 -- Forzar modo transaccional en esta sesión
9 • SET autocommit = 0;
10 • START TRANSACTION;
11
12 -- B1) Bloqueo en FICHA (id_ficha 2)
13 • UPDATE Biblioteca.FichaBibliografica
14 SET estanteria = 'ZB'
15 WHERE id_ficha = 2;
16
17 -----
18
19 -- B2) B intenta ahora tocar LIBRO (id 1), que ya bloqueó A
20 • UPDATE Biblioteca.Libro
21 SET titulo = CONCAT(titulo, '*B*')
22 WHERE id_Libro = 1;
23
24 -- Esperado: MySQL detecta el ciclo de espera y levanta:
25 -- ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
26
27 • ROLLBACK
```

The Output window shows the execution log:

#	Time	Action	Message
1	21:28:14	USE Biblioteca	0 row(s) affected
2	21:28:14	SET autocommit=0	0 row(s) affected
3	21:28:14	START TRANSACTION	0 row(s) affected
4	21:28:14	UPDATE FichasBibliografica SET estanteria = 'ZB' WHERE id_ficha = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
5	21:31:02	UPDATE Libro SET titulo = CONCAT(titulo, '*B*') WHERE id_Libro = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
6	21:35:03	ROLLBACK	0 row(s) affected
7	21:35:11	SELECT * FROM biblioteca.fichasbibliografica LIMIT 0, 1000	1000 row(s) returned
8	21:38:55	SELECT CONNECTION_ID() LIMIT 0, 1000	1 row(s) returned
9	22:07:19	SELECT DATABASE() AS actual_schema, CONNECTION_ID() AS cone_id, @@autocommit AS autocommit_status LIMIT 0, 1000	1 row(s) returned
10	22:07:19	USE Biblioteca	0 row(s) affected
11	22:07:19	SET autocommit = 0	0 row(s) affected
12	22:07:19	START TRANSACTION	0 row(s) affected
13	22:07:19	UPDATE Biblioteca.FichaBibliografica SET estanteria = 'ZB' WHERE id_ficha = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
14	22:25:58	UPDATE Biblioteca.Libro SET titulo = CONCAT(titulo, '*B*') WHERE id_Libro = 1	Error Code '1213 Deadlock' found when trying to get lock; try restarting transaction

Tabla LIBRO luego de aplicar la operación en la sesión A:

Biblioteca_Modificado fichabibliografica libro x procedure-retry tabla_log sesionA

Limit to 1000 row

1 • **SELECT * FROM biblioteca.libro;**

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: Fetch rows:

	id_Libro	eliminado	titulo	autor	editorial	anioEdicion	id_ficha
▶	1	1	Libro #1 *LOCK	Gabriel García Márquez	Anagrama	1973	1
	2	0	Libro #2	Haruki Murakami	Planeta	1832	2
	3	0	Libro #3	Stephen King	Random House	1690	3
	4	0	Libro #4	Jorge Luis Borges	Sudamericana	1549	4
	5	0	Libro #5	Gabriel García Márquez	Anagrama	1983	5
	6	0	Libro #6	Haruki Murakami	Seix Barral	1841	6
	7	0	Libro #7	Stephen King	Random House	1700	7
	8	0	Libro #8	Jorge Luis Borges	Sudamericana	1558	8
	9	0	Libro #9	Gabriel García Márquez	Anagrama	1993	9
	10	0	Libro #10	Haruki Murakami	Seix Barral	1851	10
	11	0	Libro #11	Stephen King	Random House	1710	11
	12	0	Libro #12	Jorge Luis Borges	Sudamericana	1568	12
	13	0	Libro #13	Gabriel García Márquez	Anagrama	2002	13
	14	0	Libro #14	Haruki Murakami	Seix Barral	1861	14
	15	0	Libro #15	Stephen King	Random House	1719	15
	16	0	Libro #16	Jorge Luis Borges	Sudamericana	1578	16
	17	0	Libro #17	Gabriel García Márquez	Anagrama	2012	17
	18	0	Libro #18	Haruki Murakami	Seix Barral	1871	18
	19	0	Libro #19	Stephen King	Random House	1729	19
	20	0	Libro #20	Jorge Luis Borges	Sudamericana	1587	20
	21	0	Libro #21	Gabriel García Márquez	Anagrama	2022	21
	22	0	Libro #22	Haruki Murakami	Seix Barral	1880	22
	23	0	Libro #23	Stephen King	Random House	1739	23
	24	0	Libro #24	Jorge Luis Borges	Sudamericana	1597	24

libro 8 x

Output

2) READ COMMITTED vs REPEATABLE READ

REPEATABLE READ:

También se utilizarán dos sesiones A y B.

Paso 1:

En una sesión A, se comienza una transacción definiendo el nivel de aislamiento como REPEATABLE READ.

Luego, se realiza la lectura de la tabla Libro buscando el registro para el cual ID_LIBRO = 1.

Como se puede observar, tiene el valor ELIMINADO = 0:

File Edit View Query Database Server Tools Scripting Help

Navigator: Biblioteca_Modificado fichabibliografica libro procedure-retry tabla_log repeatable-read

SCHEMAS

Filter objects

biblioteca

Tables

fichabibliografica

libro

tbllog

Views

Stored Procedures

Functions

libreria

phpmyadmin

test

```
1 USE Biblioteca;
2
3 -- repeatable read --> Las lecturas con SELECT recuperan siempre los mismos valores dentro de una misma transacción
4 -- Los cambios dentro de otras transacciones no se reflejan
5 SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
6 START TRANSACTION;
7
8 -- Leemos el estado inicial
9 SELECT id_Libro, titulo, eliminado
10 FROM Libro
11 WHERE id_Libro = 1;
12 -- → devuelve eliminado = 0
13
14
15 COMMIT;
```

Result Grid

id_Libro	titulo	eliminado
1	Libro #1	0

Libro 3

Output

#	Time	Action	Message
1	23:42:46	SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ	0 row(s) affected
2	23:42:46	START TRANSACTION	0 row(s) affected
3	23:42:46	SELECT id_Libro, titulo, eliminado FROM Libro WHERE id_Libro = 1 LIMIT 0, 1000	1 row(s) returned

Administration Schemas

Information

Connection Details

Name: facultad

Host: 127.0.0.1

Port: 3306

login

Paso 2:

En una sesión B, se realiza una modificación de la misma entrada de Libro consultada en la sesión A, y luego se ejecuta el COMMIT. Ahora ELIMINADO = 1 para el ID_LIBRO = 1.

File Edit View Query Database Server Tools Scripting Help

Navigator: sessionB query_tabla_log read-committed

SCHEMAS

Filter objects

biblioteca

Tables

fichabibliografica

libro

tbllog

Views

Stored Procedures

Functions

libreria

phpmyadmin

test

```
1 USE Biblioteca;
2
3 -- read committed --> Los SELECT recuperan sólo los valores ya commitados por otras transacciones
4 SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
5 START TRANSACTION;
6
7 UPDATE Libro
8 SET eliminado = 1
9 WHERE id_Libro = 1;
10
11 COMMIT;
```

Output

#	Time	Action	Message
1	23:44:38	USE Biblioteca	0 row(s) affected
2	23:44:38	SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED	0 row(s) affected
3	23:44:38	START TRANSACTION	0 row(s) affected
4	23:44:38	UPDATE Libro SET eliminado = 1 WHERE id_Libro = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
5	23:44:38	COMMIT	0 row(s) affected

Paso 3:

Como hemos realizado el commit en la transacción de la sesión B, al volver a la sesión A y realizar una nueva lectura vemos que a pesar de haber modificado la misma entrada de la consulta a Libro, ésta no refleja los cambios al ser leída dentro de la transacción abierta de la sesión A: el valor de ELIMINADO sigue siendo 0.

The screenshot shows a SQL Studio window with a query editor and a results grid. The query is as follows:

```
1 USE Biblioteca;
2
3 -- repeatable read --> Las lecturas con SELECT recuperan siempre los mismos valores dentro de una misma transacción
4 -- Los cambios dentro de otras transacciones no se reflejan
5 SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
6 START TRANSACTION;
7
8 -- Leemos el estado inicial
9 SELECT id_libro, titulo, eliminado
10 FROM libro
11 WHERE id_libro = 1;
12 -- devuelve eliminado = 0
13
14
15 COMMIT;
```

The results grid shows the following data:

id_libro	titulo	eliminado
1	Libro #1 "LOCK"	0

The Action Output window shows the following log:

#	Time	Action	Message
1	23:42:46	SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ	0 row(s) affected
2	23:42:46	START TRANSACTION	0 row(s) affected
3	23:42:46	SELECT id_libro, titulo, eliminado FROM libro WHERE id_libro = 1 LIMIT 0, 1000	1 row(s) returned
4	23:43:32	SELECT * FROM biblioteca.libro LIMIT 0, 1000	1000 row(s) returned
5	23:46:15	SELECT id_libro, titulo, eliminado FROM libro WHERE id_libro = 1 LIMIT 0, 1000	1 row(s) returned

Paso 4:

Sólo luego de ejecutar el COMMIT que da cierre a la transacción de la sesión A se ven reflejados los cambios de la sesión B al realizar nuevamente la consulta, ya que de esta manera hemos dado cierre a la transacción.

Con esto podemos evidenciar cómo las consultas aisladas bajo REPEATABLE READ únicamente toman los valores presentes al momento de iniciada la transacción, más no reflejan los cambios hechos por otras transacciones hasta el momento en que ésta es finalizada con un COMMIT que confirme los cambios o un ROLLBACK.

The screenshot shows the same SQL Studio window as in Paso 3, but now the transaction has been committed. The query results are as follows:

id_libro	titulo	eliminado
1	Libro #1 "LOCK"	1

The Action Output window shows the following log:

#	Time	Action	Message
1	23:42:46	SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ	0 row(s) affected
2	23:42:46	START TRANSACTION	0 row(s) affected
3	23:42:46	SELECT id_libro, titulo, eliminado FROM libro WHERE id_libro = 1 LIMIT 0, 1000	1 row(s) returned
4	23:43:32	SELECT * FROM biblioteca.libro LIMIT 0, 1000	1000 row(s) returned
5	23:46:15	SELECT id_libro, titulo, eliminado FROM libro WHERE id_libro = 1 LIMIT 0, 1000	1 row(s) returned
6	23:49:31	COMMIT	0 row(s) affected
7	23:49:47	SELECT id_libro, titulo, eliminado FROM libro WHERE id_libro = 1 LIMIT 0, 1000	1 row(s) returned

READ COMMITTED:

Para evidenciar un READ COMMITTED, primeramente revertiremos los cambios realizados en Libro y cambiaremos el nivel de aislamiento de la lectura en la sesión A por READ COMMITTED.

Paso 1:

En sesión A: se inicializa la transacción y se realiza la lectura de Libro tal como se hizo antes: vemos que el valor ELIMINADO vuelve a ser 0 de manera inicial

The screenshot displays a MySQL IDE interface with the following components:

- Navigator:** Shows the database schema 'biblioteca' with tables 'fichabibliografica', 'libro', and 'blog'.
- SQL Editor:** Contains the following SQL script:

```
1 USE Biblioteca;
2
3 -- limpio posibles transacciones abiertas
4 ROLLBACK;
5 COMMIT;
6
7 -- repeatable read --> Las lecturas con SELECT recuperan siempre los mismos valores dentro de una misma transacción
8 -- Los cambios dentro de otras transacciones no se reflejan
9 SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
10
11 SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
12 START TRANSACTION;
13
14 -- Leemos el estado inicial
15 SELECT id_Libro, titulo, eliminado
16 FROM Libro
17 WHERE id_Libro = 1;
18
19
20 COMMIT;
```
- Result Grid:** Displays the result of the SELECT query:

id_Libro	titulo	eliminado
1	Libro #1	0
- Output Window:** Shows the execution steps and the SQL statement executed:

#	Time	Action	Message
1	00:27:40	USE Biblioteca	0 row(s) affected
2	00:27:40	ROLLBACK	0 row(s) affected
3	00:27:40	COMMIT	0 row(s) affected
4	00:27:40	SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED	0 row(s) affected
5	00:27:40	START TRANSACTION	0 row(s) affected
6	00:27:40	SELECT id_Libro, titulo, eliminado FROM Libro WHERE id_Libro = 1 LIMIT 0, 1000	1 row(s) returned
- Connection Details:** Shows the connection information for the 'facultad' database on host '127.0.0.1' at port '3306'.

Paso 2:

Pasando a la sesión B, comenzamos otra transacción y realizamos la modificación de Libro otra vez con el UPDATE sobre el mismo registro consultado en A.

ELIMINADO = 1 para ID_LIBRO = 1.

Ejecutamos el COMMIT al final para cerrar la transacción en la sesión B:

facultad (biblioteca) - Warning x

File Edit View Query Database Server Tools Scripting Help

Navigator: sessionB query_tabla_log libro read-committed

SCHMAS

Filter objects

biblioteca

Tables

fichabibliografica

libro

blog

Views

Stored Procedures

Functions

libreria

phpmyadmin

test

```

1 • USE Biblioteca;
2
3 -- read committed --> Los SELECT recuperan sólo los valores ya commitados por otras transacciones
4 -- SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
5 START TRANSACTION;
6
7 • UPDATE Libro
8 SET eliminado = 1
9 WHERE id_Libro = 1;
10
11 • COMMIT;

```

Output

Action Output

#	Time	Action	Message
1	00:30:20	USE Biblioteca	0 row(s) affected
2	00:30:20	START TRANSACTION	0 row(s) affected
3	00:30:20	UPDATE Libro SET eliminado = 1 WHERE id_Libro = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
4	00:30:20	COMMIT	0 row(s) affected

Paso 3:

Volviendo a la sesión A, y haciendo la consulta nuevamente, vemos que sin aplicar el COMMIT podemos ahora ver reflejados los cambios realizados por la transacción de la sesión B, a diferencia de lo que ocurría al realizar la lectura con el nivel de aislamiento REPEATABLE READ de la primer prueba:

facultad (biblioteca) - Warn... x

File Edit View Query Database Server Tools Scripting Help

Navigator: Biblioteka_Modificado fichabibliografica libro procedure-retry tabla_log repeatable-read x

SCHMAS

Filter objects

biblioteca

Tables

fichabibliografica

libro

blog

Views

Stored Procedures

Functions

libreria

phpmyadmin

test

```

1 USE Biblioteca;
2
3 -- limpio posibles transacciones abiertas
4 • ROLLBACK;
5 • COMMIT;
6
7 -- repeatable read --> Las lecturas con SELECT recuperan siempre los mismos valores dentro de una misma transacción
8 -- Los cambios dentro de otras transacciones no se reflejan
9 -- SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
10
11 • SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
12 • START TRANSACTION;
13
14 -- Leemos el estado inicial
15 • SELECT id_Libro, titulo, eliminado
16 FROM Libro
17 WHERE id_Libro = 1;
18
19
20 • COMMIT;

```

Result Grid

id_Libro	titulo	eliminado
1	Libro #1 "LOCK	1

Libro 10 x

Output

Action Output

#	Time	Action	Message
1	00:27:40	USE Biblioteca	0 row(s) affected
2	00:27:40	ROLLBACK	0 row(s) affected
3	00:27:40	COMMIT	0 row(s) affected
4	00:27:40	SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED	0 row(s) affected
5	00:27:40	START TRANSACTION	0 row(s) affected
6	00:27:40	SELECT id_Libro, titulo, eliminado FROM Libro WHERE id_Libro = 1 LIMIT 0, 1000	1 row(s) returned
7	00:31:14	SELECT id_Libro, titulo, eliminado FROM Libro WHERE id_Libro = 1 LIMIT 0, 1000	1 row(s) returned

Administration Schemas

Information

Connection Details

Name: facultad

Host: 127.0.0.1

Port: 3306

Login: root

User: root

Current User: root@localhost

SSL: SSL not used

Server

Con esto evidenciamos que utilizando el nivel de aislamiento READ COMMITTED podemos ver reflejados los cambios a nivel base de datos realizados por otras transacciones en tiempo de ejecución sin haber cerrado la de la sesión actualmente ejecutada.

3) Retry Procedure + Log de errores

En la siguiente prueba, reproduciremos un bloqueo luego de ejecutar varios intentos de acceso mediante loops definidos en lenguaje SQL dentro de un procedure almacenado en un script:

Prueba sin errores:

Tabla con log de errores definida:

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'biblioteca' expanded. The center pane shows the SQL script for creating the 'TxLog' table. The right pane shows the 'Output' window with a warning message.

```
1 USE Biblioteca;
2
3 CREATE TABLE IF NOT EXISTS TxLog (
4     id BIGINT PRIMARY KEY AUTO_INCREMENT,
5     ts TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
6     sesion BIGINT NOT NULL,
7     codigo INT NULL,
8     sql_state VARCHAR(5) NULL,
9     mensaje VARCHAR(500) NULL,
10    detalle VARCHAR(500) NULL
11 ) ENGINE=InnoDB;
```

#	Time	Action	Message
1	00:47:26	USE Biblioteca	0 row(s) affected
2	00:47:26	CREATE TABLE IF NOT EXISTS TxLog (id BIGINT PRIMARY KEY AUTO_INCREMENT, ts TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, sesion BIGINT NOT NULL, codigo INT NULL, sql_state VARCHAR(5) NULL, mensaje VARCHAR(500) NULL, detalle VARCHAR(500) NULL) ENGINE=InnoDB;	0 row(s) affected, 1 warning(s): 1050 Table 'tbllog' already exists

Aquí se almacenarán los errores registrados

- Id: número del error
- Ts: timestamp (fecha y hora del error)
- Sesión: registra el ID de la sesión
- Código: código de error
- Sql_stat: status de la ejecución
- Mensaje: mensaje de error
- Detalle: detalle del error

Se utiliza el motor InnoDB para hacerlo compatible con ejecución de transacciones, bloque de filas y recuperación ante fallos

Ejecución del procedure (ver código completo en zip adjunto/repositorio):

The screenshot displays the MySQL Workbench interface. The Navigator pane on the left shows the database schema with the following structure:

- biblioteca
 - Tables
 - fichabibliografica
 - libro
 - blog
 - Views
 - Stored Procedures
 - Functions
 - libreria
 - phmyadmin
 - test

The central SQL editor shows the code for the stored procedure `sp_tx_biblioteca_segura`. The code includes comments in Spanish and SQL statements for creating, dropping, and executing the procedure. The procedure takes three parameters: `p_id_libro` (INT), `p_nueva_estanteria` (VARCHAR(20)), and `p_nuevo_idioma` (VARCHAR(30)). It declares several local variables and uses a `REPEAT` loop with a `DECLARE CONTINUE HANDLER` for errors to handle retries.

The bottom pane shows the execution results. The `Result Grid` displays the following data:

etapa	ROUTINE_SCHEMA	ROUTINE_NAME
OK_procedure	biblioteca	sp_tx_biblioteca_segura

The `Output` pane shows the execution log with the following messages:

- 1 16:40:56 USE Biblioteca; 0 row(s) affected
- 2 16:40:56 DROP PROCEDURE IF EXISTS sp_tx_biblioteca_segura; 0 row(s) affected
- 3 16:40:56 -- Firma del procedimiento que recibe: CREATE PROCEDURE sp_tx_biblioteca_segura(IN p_id_libro INT, -- libro a modificar IN p_nueva_estanteria VARCHAR(20)-- nu... 0 row(s) affected
- 4 16:40:56 -- Chequeo rápido SELECT 'OK_procedure' AS etapa, ROUTINE_SCHEMA, ROUTINE_NAME FROM INFORMATION_SCHEMA.ROUTINES WHERE ROUTINE_NAME='sp... 1 row(s) returned

Llamado al procedure:

The screenshot displays the MySQL Workbench interface. The Navigator pane on the left shows the database schema with the following structure:

- biblioteca
 - Tables
 - fichabibliografica
 - libro
 - blog
 - Views
 - Stored Procedures
 - Functions
 - libreria
 - phmyadmin
 - test

The central SQL editor shows the query used to call the stored procedure `sp_tx_biblioteca_segura`. The query includes comments in Spanish and SQL statements for creating, dropping, and executing the procedure. The procedure takes three parameters: `p_id_libro` (INT), `p_nueva_estanteria` (VARCHAR(20)), and `p_nuevo_idioma` (VARCHAR(30)). It declares several local variables and uses a `REPEAT` loop with a `DECLARE CONTINUE HANDLER` for errors to handle retries.

The bottom pane shows the execution results. The `Result Grid` displays the following data:

ts	sesion	codigo	sql_state	mensaje	detalle
16:43:38					

The `Output` pane shows the execution log with the following messages:

- 1 16:40:56 USE Biblioteca; 0 row(s) affected
- 2 16:40:56 DROP PROCEDURE IF EXISTS sp_tx_biblioteca_segura; 0 row(s) affected
- 3 16:40:56 -- Firma del procedimiento que recibe: CREATE PROCEDURE sp_tx_biblioteca_segura(IN p_id_libro INT, -- libro a modificar IN p_nueva_estanteria VARCHAR(20)-- nu... 0 row(s) affected
- 4 16:40:56 -- Chequeo rápido SELECT 'OK_procedure' AS etapa, ROUTINE_SCHEMA, ROUTINE_NAME FROM INFORMATION_SCHEMA.ROUTINES WHERE ROUTINE_NAME='sp... 1 row(s) returned
- 5 16:43:38 CALL sp_tx_biblioteca_segura(1, 'E7', 'Español'); 2 row(s) affected
- 6 16:43:38 SELECT id_ficha, estanteria, idioma FROM Fichabibliografica WHERE id_ficha = 1 LIMIT 0, 1000 1 row(s) returned
- 7 16:43:38 SELECT ts, sesion, codigo, sql_state, mensaje, detalle FROM TxLog ORDER BY id DESC LIMIT 5 0 row(s) returned

Accediendo a la tabla de errores, vemos que no hay ninguno:

facultad (biblioteca) - Warn... x

facultad - Warning - not suppo.x

File Edit View Query Database Server Tools Scripting Help

Navigator

filter objects

biblioteca

Tables

fichabibliografica

libro

tbllog

Views

Stored Procedures

Functions

libreria

phpmyadmin

test

Biblioteca_Modificado

fichabibliografica

libro

procedure-retry*

tabla_log

prueba1-sin-error

tbllog

1

SELECT

FROM biblioteca.tbllog;

Limit to 1000 row

✱

🔍

📄

🔗

Result Grid

Filter Rows:

Edit

Export/Import:

Wrap Cell Content:

	id	ts	sesion	codigo	sql_state	mensaje	detalle
	INDEX	INDEX	INDEX	INDEX	INDEX	INDEX	INDEX

tbllog 2 x

Output

Action Output

#	Time	Action	Message
1	16:40:56	USE Biblioteca	0 row(s) affected
2	16:40:56	DROP PROCEDURE IF EXISTS sp_tx_biblioteca_segura	0 row(s) affected
3	16:40:56	-- Firma del procedimiento que recibe: CREATE PROCEDURE sp_tx_biblioteca_segura(IN p_id_libro INT, -- libro a modificar IN p_nueva_estanteria VARCHAR(20),-- nu...	0 row(s) affected
4	16:40:56	-- Chequeo rápido SELECT 'OK_procedure' AS etapa, ROUTINE_SCHEMA, ROUTINE_NAME FROM INFORMATION_SCHEMA.ROUTINES WHERE ROUTINE_NAME='sp...	1 row(s) returned
5	16:43:38	CALL sp_tx_biblioteca_segura(1,'E7','Español')	2 row(s) affected
6	16:43:38	SELECT id_ficha, estanteria, idioma FROM FichaBibliografica WHERE id_ficha = 1 LIMIT 0, 1000	1 row(s) returned
7	16:43:38	SELECT ts, sesion, codigo, sql_state, mensaje, detalle FROM TxLog ORDER BY id DESC LIMIT 5	0 row(s) returned
8	16:44:49	SELECT * FROM biblioteca.tbllog LIMIT 0, 1000	0 row(s) returned
9	16:44:52	SELECT * FROM biblioteca.tbllog LIMIT 0, 1000	0 row(s) returned

PRUEBA CON ERRORES:

Paso 1: Se bloque Libro en sesión A:

facultad (biblioteca) - Warn... x

facultad (biblioteca) - Warning.x

File Edit View Query Database Server Tools Scripting Help

Navigator

filter objects

biblioteca

Tables

fichabibliografica

libro

tbllog

Views

Stored Procedures

Functions

libreria

phpmyadmin

test

Biblioteca_Modificado

fichabibliografica

libro

procedure-retry

tabla_log

prueba1-sin-error

sesionA

script_generacion_1000_regi...

1

USE Biblioteca;

2

3

SET autocommit = 0;

4

START TRANSACTION;

5

6

-- Bloqueamos el libro 1 (queda "ocupado" hasta hacer COMMIT/ROLLBACK)

7

UPDATE Libro

8

SET titulo = CONCAT(titulo, ' *LOCK')

9

WHERE id_Libro = 1;

10

11

-- No se realiza todavía el COMMIT

12

SELECT CONNECTION_ID() AS connA, 'A: bloqueado activo' AS estado;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	connA	estado
10	A:	bloqueado activo

Result 4 x

Output

Action Output

#	Time	Action	Message
1	22:33:32	USE Biblioteca	0 row(s) affected
2	22:33:32	CREATE TABLE IF NOT EXISTS TxLog (id BIGINT PRIMARY KEY AUTO_INCREMENT, ts TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, sesion ...	0 row(s) affected. 1 warning(s): 1050 Table 'tbllog' already exists
3	22:33:42	USE Biblioteca	0 row(s) affected
4	22:33:42	DROP PROCEDURE IF EXISTS sp_tx_biblioteca_segura	0 row(s) affected
5	22:33:42	CREATE PROCEDURE sp_tx_biblioteca_segura(IN p_id_libro INT, -- libro a modificar IN p_nueva_estanteria VARCHAR(20), -- nuevo nro de estanteria IN p_nu...	0 row(s) affected
6	22:38:20	SET autocommit = 0	0 row(s) affected
7	22:38:20	START TRANSACTION	0 row(s) affected
8	22:38:20	UPDATE Libro SET titulo = CONCAT(titulo, ' *LOCK') WHERE id_Libro = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
9	22:38:20	SELECT CONNECTION_ID() AS connA, 'A: bloqueado activo' AS estado LIMIT 0, 1000	1 row(s) returned

Administration

Schemas

Information

Connection Details

Name: facultad

Host: 127.0.0.1

Port: 3306

Login User: root

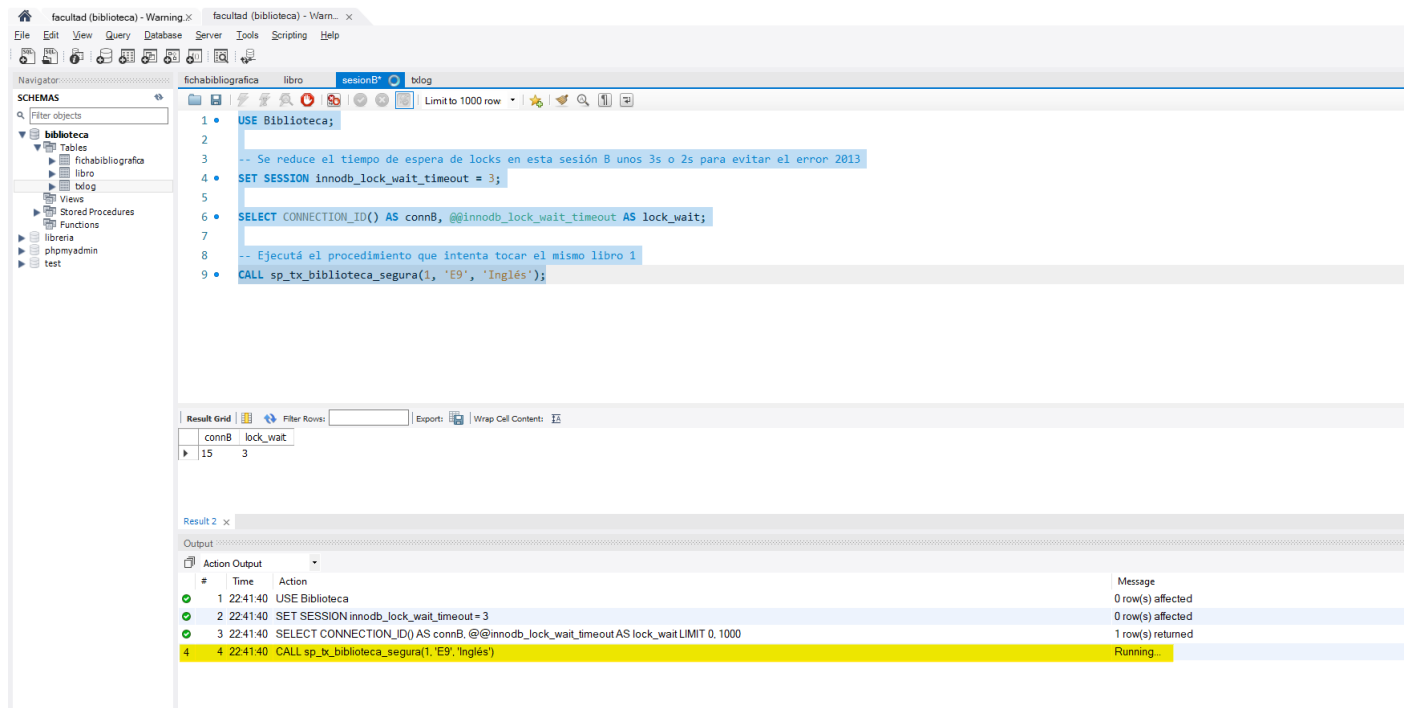
Current User: root@localhost

SSL cipher: SSL not used

Paso 2:

Se llama al procedimiento que accede al mismo recurso en la sesión B.

Como el recurso está bloqueado en la sesión A, la transacción en B queda en suspenso durante unos cuantos segundos, según se ha definido en el procedure:

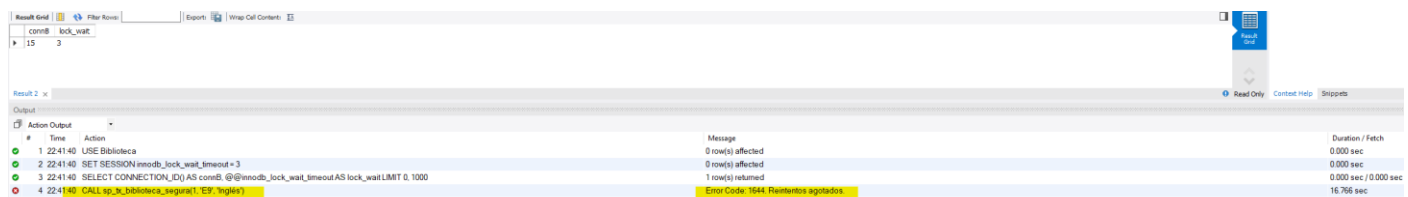


```
1 USE Biblioteca;
2
3 -- Se reduce el tiempo de espera de locks en esta sesión B unos 3s o 2s para evitar el error 2013
4 SET SESSION innodb_lock_wait_timeout = 3;
5
6 SELECT CONNECTION_ID() AS connB, @@innodb_lock_wait_timeout AS lock_wait;
7
8 -- Ejecuté el procedimiento que intenta tocar el mismo libro 1
9 CALL sp_tx_biblioteca_segura(1, 'E9', 'Inglés');
```

connB	lock_wait
15	3

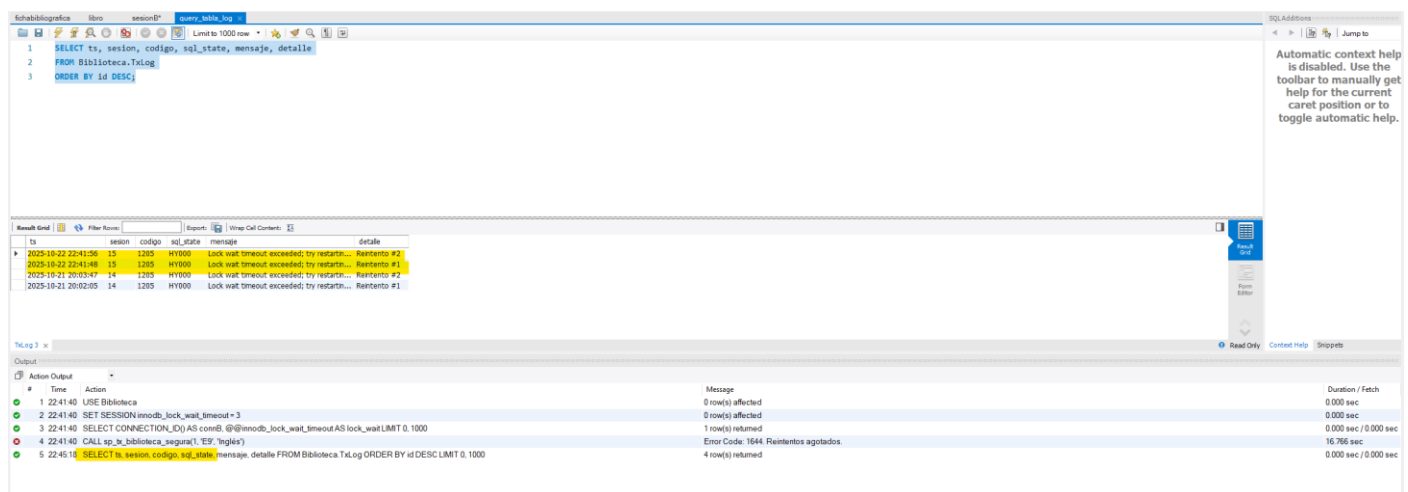
#	Time	Action	Message
1	22:41:40	USE Biblioteca	0 row(s) affected
2	22:41:40	SET SESSION innodb_lock_wait_timeout = 3	0 row(s) affected
3	22:41:40	SELECT CONNECTION_ID() AS connB, @@innodb_lock_wait_timeout AS lock_wait LIMIT 0, 1000	1 row(s) returned
4	22:41:40	CALL sp_tx_biblioteca_segura(1, 'E9', 'Inglés')	Running...

Luego de unos segundos, la ejecución resulta en un error 1644 por reiterados intentos fallidos:



#	Time	Action	Message	Duration / Fetch
1	22:41:40	USE Biblioteca	0 row(s) affected	0.000 sec
2	22:41:40	SET SESSION innodb_lock_wait_timeout = 3	0 row(s) affected	0.000 sec
3	22:41:40	SELECT CONNECTION_ID() AS connB, @@innodb_lock_wait_timeout AS lock_wait LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
4	22:41:40	CALL sp_tx_biblioteca_segura(1, 'E9', 'Inglés')	Error Code: 1644 Reintentos agotados	16.766 sec

Accediendo a la tabla de logs, veremos una serie de errores por varios reintentos ejecutados (dos de ellos corresponden a ejecuciones anteriores, como podemos ver en el timestamp):



```
1 SELECT ts, sesion, codigo, sql_state, mensaje, detalle
2 FROM Biblioteca.TxLog
3 ORDER BY id DESC;
```

ts	sesion	codigo	sql_state	mensaje	detalle
2025-10-22 22:41:56	15	1205	HY000	Lock wait timeout exceeded; try restarting...	Reintento #2
2025-10-22 22:41:49	15	1205	HY000	Lock wait timeout exceeded; try restarting...	Reintento #1
2025-10-21 20:02:47	14	1205	HY000	Lock wait timeout exceeded; try restarting...	Reintento #2
2025-10-21 20:02:05	14	1205	HY000	Lock wait timeout exceeded; try restarting...	Reintento #1

#	Time	Action	Message	Duration / Fetch
1	22:41:40	USE Biblioteca	0 row(s) affected	0.000 sec
2	22:41:40	SET SESSION innodb_lock_wait_timeout = 3	0 row(s) affected	0.000 sec
3	22:41:40	SELECT CONNECTION_ID() AS connB, @@innodb_lock_wait_timeout AS lock_wait LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
4	22:41:40	CALL sp_tx_biblioteca_segura(1, 'E9', 'Inglés')	Error Code: 1644 Reintentos agotados	16.766 sec
5	22:45:14	SELECT ts, sesion, codigo, sql_state, mensaje, detalle FROM Biblioteca.TxLog ORDER BY id DESC LIMIT 0, 1000	4 row(s) returned	0.005 sec / 0.000 sec

ANEXO:

Se ha utilizado el soporte de la IA, utilizando chatGPT 5 para realizar correcciones y mejoras al código, sobre todo en la parte del armado del procedure que aplica los reintentos de acceso, que fue la parte que más complicada se me presentó al momento de resolver la consigna:

Según el Workbench, hay un problema con la posición del primer PROCEDURE: dice que espera AGGREGATE, ALGORITHM, DATABASE, DEFINER o alguno de otras palabras clave que lista...

Pensó durante 29s >

Eso suele pasar por **dos cosas** (a veces las dos a la vez):

1. Tu versión de MySQL no admite `CREATE OR REPLACE PROCEDURE`

En varias versiones 8.0 (y todas las 5.7) *no* existe "OR REPLACE" para procedimientos.

El parser ve `CREATE OR ...` y te dice "esperaba `DEFINER`, `DATABASE`, `ALGORITHM`, etc."

👉 **Solución portable:** usá `DROP PROCEDURE IF EXISTS` + `CREATE PROCEDURE` (sin OR REPLACE).

2. Falta fijar el delimitador antes de definir el procedimiento

Si no ponés `DELIMITER $$` (o similar) *antes* del `CREATE PROCEDURE`, los `;` internos cortan la definición y el parser "se desarma".

👉 **Solución:** poné `DELIMITER $$` antes del `CREATE`, y cerrá con `END$$` y `DELIMITER ;` al final.

Bloque compatible (portátil) listo para pegar

Esto evita "OR REPLACE" y fija bien los delimitadores. No cambia tu lógica.

```
sql Copiar código

USE Biblioteca;

-- 1) Eliminalo si existe (en vez de OR REPLACE)
DROP PROCEDURE IF EXISTS sp_tx_biblioteca_segura;

-- 2) Cambiamos el delimitador para que el cuerpo pueda usar ';'
DELIMITER $$

CREATE PROCEDURE sp_tx_biblioteca_segura(
    IN p_id_libro INT,
    IN p_nueva_estanteria VARCHAR(20),
    IN p_nuevo_idioma VARCHAR(30)
)
BEGIN
```

+ |pregunta lo que quieras



```

-- 2) Actualiza Libro
UPDATE Libro
SET eliminado = 0
WHERE id_Libro = p_id_libro;

-- 3) Actualiza Ficha
UPDATE FichaBibliografica
SET estanteria = p_nueva_estanteria,
    idioma      = p_nuevo_idioma
WHERE id_ficha = @v_ficha;

COMMIT;
END;

IF v_done = 0 THEN
    SET v_tries = v_tries + 1;
    DO SLEEP(0.25 * v_tries); -- backoff: 0.25s, 0.5s
END IF;

UNTIL v_done = 1 OR v_tries >= 2 END REPEAT;

IF v_done = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Reintentos agotados.';
END IF;
END$$

-- 3) Volvemos al delimitador normal
DELIMITER ;

```

Checklist rápido (para que no falle)

- Ejecutá `USE Biblioteca;` antes (en **cada** sesión) ✓
- Asegurate de tener la tabla `TxLog` creada ✓
- **No** uses `CREATE OR REPLACE PROCEDURE` si tu versión no lo soporta ✓
- Sí usá `DELIMITER $$` antes y `DELIMITER ;` después ✓

Podés chequear tu versión con:

```

sql
SELECT VERSION();

```

+ Pregunta lo que quieras

Por cuestiones de practicidad, no pegaré todas las consultas, ya que los hilos en ocasiones se hicieron muy largos.